/websearch

Show or test web search engines (/websearch show|test|help)

Overview

`/websearch` inspects the MasterFetch search pipeline used by `mf_search`. With no arguments it behaves like `show`, printing a table of the configured search engines and whether each is enabled and currently in use. `test` launches an asynchronous diagnostic run whose result is delivered into the message window once it finishes.

Syntax

```
/websearch          Show the engine status table (same as show)
/websearch show     Show the engine status table
/websearch test     Run an asynchronous engine diagnostic
/websearch help     Show the help table
```

Options / Subcommands

Form	Description
`/websearch`	Alias of `show`
`/websearch show`	Prints the `Engine / Enabled / In Use / Failed` status table
`/websearch test`	Runs the diagnostic off the UI thread; the result arrives later and drains into the message window
`/websearch help`	Prints the help table

Engine table

The `show` table lists, per engine: whether it is enabled, whether it is currently selected for use, and whether the last diagnostic run marked it as failed.

- Keyless engines that need no API key: DuckDuckGo, Brave, and Google (Google extraction uses headless Chrome).
- API-backed engines activate only when a key is configured: `langsearch_api_key`, and `tavily_api_key` (or the `TAVILY_API_KEY` environment variable), and `perplexity_api_key` (or `PERPLEXITY_API_KEY`). Configure these in `ragent.json`.

Examples

`/websearch`

Prints the engine status table.

`/websearch show`

Identical to the bare form.

`/websearch test`

Starts the diagnostic in the background; a status line confirms the run began, and the per-engine results are appended to the message window when the run finishes.

`/websearch help`

Prints the help table.

Output

- Bare form and `show`: the engine status table plus notes on which engines are keyless and which require API keys.
- `test`: an immediate confirmation that the diagnostic started, then the diagnostic result later (the result is queued and drained on a later poll, so it can appear after subsequent activity).
- Unknown subcommand: a warning message.

Related

- `mf_search` - the agent tool the engines back
- `mf_fetch`, `mf_crawl`, `mf_cache_clear` - the rest of the MasterFetch tool family
- `/webapi` - control the built-in HTTP API server
- `ragent.json` - where `langsearch_api_key`, `tavily_api_key`, and `perplexity_api_key` are configured